Listing 12.12 Processing the `Settle` message

```
object Bet {
  ...

  private def settle(
      state: State,
      command: Settle,
      sharding: ClusterSharding,
      context: ActorContext[Command]): Effect[Event, State] = {
    implicit val timeout = Timeout(10, SECONDS)                          ❶

    def auxCreateRequest(stake: Int)(                                    ❷
        replyTo: ActorRef[Wallet.Response]): Wallet.AddFunds =
      Wallet.AddFunds(stake, replyTo)

    if (isWinner(state, command.result)) {                               ❸
      val walletRef =
        sharding.entityRefFor(Wallet.typeKey, state.status.walletId)     ❹
      context.ask(walletRef, auxCreateRequest(state.status.stake)) {     ❺
        case Success(_) =>
          Close(s"stake reimbursed to wallet [$walletRef]")
        case Failure(ex) => //I rather retry
          val message =
            s"state NOT reimbursed to wallet [$walletRef]. Reason
            ➥[${ex.getMessage}]"
          context.log.error(message)
          Fail(message)
      }
    }
    Effect.none                                                          ❻
  }
  ...
}
```

❶ Ask timeout

❷ Auxiliary method to produce an ask message with parameters, apart from replyTo

❸ Checks whether the bet chose the winning result

❹ Finds the wallet ActorRef

❺ Asks the wallet to add the winnings

❻ Doesn’t persist any event